MNRAS **000**, 1–5 (2026) Preprint 12 August 2026 Compiled using MNRAS LATEX style file v3.2

A dependency-injection port of the STEEL statistical semi-empirical model to Rust, and a three-way validation against the published results

Philip J. Grylls[1]⋆
[1]*Department for Physics and Astronomy, University of Southampton, Highfield SO17 1BJ, UK*

12 August 2026

ABSTRACT

STEEL (Statistical sEmi-Empirical modeL) is the dark-matter-accretion-history-based semi-empirical galaxy formation model behind three papers (Grylls et al. 2019, 2020a,b). The original code is a mix of Python and hand-tuned Cython that no longer runs on a current software stack: it depends on removed NUMPY/SCIPY/PANDAS APIs, a machine-specific compiled Fortran binary for the halo mass accretion history, and an interactive confirmation prompt that blocks unattended execution. We describe a from-scratch, dependency-injected Rust reimplementation (`rs-steel`) built around one trait per physical process – cosmology, halo growth, halo and subhalo mass functions, merger time-scale, stellar–halo mass relation, star formation, quenching, and stellar stripping – and validate it against both the original code and a corrected version of it (`py-corrected`) on every headline results figure across all three papers. In deterministic mode (scatter disabled on both sides) every integrated quantity we compare agrees to better than 0.5% between `py-corrected` and `rs-steel`, with disagreement concentrated in the sparse high-mass tail where individual bins hold at most one object. The reimplementation runs 4.5× faster than the (already Cython-accelerated) Python on the published parameter grid. In the process of building and validating this reproduction we found and fixed a number of defects in the original code, the most consequential of which silently zeroed the pair-fraction output in every configuration Papers 2 and 3 actually use. We give the full list, quantify which published results move and by how much, and argue that a from-scratch, independently-typed reimplementation validated this way is a practical and under-used check on semi-empirical model code.

Key words: methods: numerical – methods: statistical – galaxies: haloes – galaxies: statistics

1 INTRODUCTION

Semi-empirical models connect dark matter haloes to galaxies through observationally calibrated relations (typically abundance matching) rather than the coupled hydrodynamics and sub-grid physics of a cosmological simulation, or the merger-tree bookkeeping of a semi-analytic model. STEEL (Grylls et al. 2019) took this a step further, replacing discrete dark matter merger trees with a *statistical* accretion history: at every redshift step, the halo mass function and an analytic mass-accretion-history model together give the expected number density and mass of subhaloes accreted onto a halo of any given mass, with no dependence on simulation volume or resolution. Grylls et al. (2020a) extended this to derive star formation histories self-consistently from the growth of the dark matter and the abundance-matched stellar mass, and Grylls et al. (2020b) used the same machinery to study how the choice of stellar mass estimator propagates into the inferred galaxy pair fraction.

Software written for one project rarely survives contact with a second one unmodified, and STEEL is no exception. Re-running the published code seven years on surfaces three separate problems. First, *it does not run*: the code imports PANDAS, NUMPY and SCIPY APIs removed in the intervening major versions (§2), and the halo mass accretion history is computed by shelling out to a Fortran binary compiled with a machine-specific output path. Second, where it does run, several code paths silently compute something other than what the papers describe – not edge cases, but the exact configurations the published figures use (§5.2). Third, there is no independent implementation to check any of this against: a bug shared between the code and the papers built from it is invisible from inside that one codebase.

We address this by writing a second, independent implementation from the model description in the three papers, in a different language, with a different type system and a different numerical stack, and comparing the two on every result each paper reports. Agreement is evidence the port (and, to the extent they overlap, the original) is doing what the papers say; disagreement identifies either a porting error or a defect in the original that the comparison itself localises. This letter reports that comparison. §2 documents the repro-

⋆ E-mail: p.grylls@soton.ac.uk

ducibility failure concretely. §3 summarises the Rust implementation. §4 describes the three-way validation methodology. §5 gives the results: numerical fidelity, the defects found and their impact, wall-clock performance, and a figure-by-figure reproduction of the three papers' results. §6 discusses what generalises.

2 THE ORIGINAL CODE NO LONGER RUNS

Attempting to execute `STEEL.py` as published fails at import time: `CentralPostprocessing.py` constructs an SDSS-comparison object as a module-level side effect, so post-processing cannot be imported without first obtaining the observational catalogue, which is not distributed with the code. Past that point, `Scripts/CentralPostprocessing.py` calls `pandas.Series.get_values()` (removed in pandas 1.0), `scipy.interpolate.interp2d` and `scipy.integrate.cumtrapz` (removed in SciPy 1.14), and reads whitespace-delimited files with the `delim_whitespace` keyword (removed in pandas 2.2). None of these are exotic APIs; they are mainstream functions removed by ordinary version churn over seven years. Separately, `Functions.py::Halogrowth` obtains the halo mass accretion history by shelling out to `getPWGH`, a compiled Fortran binary invoked with a hardcoded, machine-specific output path – it has no standalone Python entry point, so nothing that calls `Halogrowth` runs on a machine that is not the original author's. `STEEL.py` itself blocks on an interactive confirmation prompt before running, precluding unattended execution. We built two pinned virtual environments (one on the last Python/NumPy/SciPy/pandas versions that still provide the removed APIs, one modern) and patched around each of these individually to obtain a runnable baseline; that baseline, not the repository as published, is what we call `py-as-is` throughout.

3 ARCHITECTURE

`rs-steel` is organised as one Rust trait per physical process rather than as a monolithic function: `Cosmology`, `HaloGrowthModel`, `HaloMassFunctionModel`, `SubhaloMassFunctionModel`, `MergerTimescaleModel`, `HaloStrippingModel`, `SmhmModel`, `SfrModel`, `QuenchingModel`, `GasMassModel`, and `StellarStrippingModel`. A TOML run file selects one implementation of each trait and a command-line orchestrator wires them together, so that changing, for example, the stellar-mass–halo-mass relation from the PyMorph (Sérsic+exponential) to the cmodel (de Vaucouleurs) calibration used throughout Grylls et al. (2020a) is a one-line configuration change rather than a code change. The workspace splits into `steel-core` (traits and orchestration), `steel-plugins` (the physical implementations), `steel-io` (run-file parsing and `.npy` output, byte-compatible with the Python's NUMPY serialisation), `steel-cli`, `steel-fit`, and `steel-postprocess`.

Several components were re-derived natively rather than ported line by line, because the originals depend on external packages (COLOSSUS, `hmf`) or the machine-specific Fortran binary described above: the Eisenstein & Hu (1998) transfer function and $\sigma(M)$ mass variance, the linear growth factor, the Despali et al. (2016) halo mass function, and a native root-find of the van den Bosch (2014) mass-accretion-history model. The last of these was checked against a freshly compiled `getPWGH` run under a matched cosmology and agrees to 0.0021 dex over $\log M_0 = 11$–15, which sets the floor on any subsequent py/rs comparison.

4 VALIDATION METHODOLOGY

We compare three implementations on identical inputs: `py-as-is` (the code as published, patched only as necessary to run, described above); `py-corrected` (`py-as-is` with the defects of §5.2 fixed, as a reviewable commit series); and `rs-steel`. Two modes answer different questions. In *deterministic* mode, scatter is disabled in both the Python and the Rust (`py-as-is` cannot participate here – its gas-mass step scatters unconditionally, Appendix A7 of the online defect log); both implementations then evaluate the same arithmetic on the same grid, so agreement is a strong, floating-point-level claim about the port itself. In *stochastic* mode, scatter is enabled and all three draw from unrelated random number generators (NUMPY's Mersenne Twister, GSL's `taus`, Rust's ChaCha), so only ensemble statistics over several seeds are comparable; this mode instead isolates what the defect fixes change, independent of any port. Reporting one blended number across both modes would conflate numerical port fidelity with Monte Carlo noise, so we keep them separate throughout.

We additionally reproduce every results figure in all three papers under this framework (§5.4), reading each figure's exact parameter configuration back out of the plotting and post-processing scripts that produced it – these configurations are not otherwise documented anywhere in the repository – and re-deriving the model curves each figure shows across all three implementations, without the external observational overlays (SDSS, Illustris TNG, Mundy et al. 2017), since matching each other is the claim this letter makes; matching the sky is the original papers' claim and is not re-litigated here.

5 RESULTS

5.1 Numerical fidelity

On the published parameter grid ($\log M_h = 11$–16.6 in 0.1 dex steps, 190 redshift steps), every integrated quantity we compare between `py-corrected` and `rs-steel` in deterministic mode agrees to better than 0.5% (reverse-cumulative comparison along the stellar mass axis; a bin-by-bin comparison of two independently binned histograms is dominated by mass moving across a bin edge, which is not what either paper's figures plot). The satellite stellar mass function median deviation is 0.93%, with the tail out to $p90 = 23.9\%$ carried entirely by the sparse high-mass end, where a bin holds at most one satellite in either implementation. Table 1 gives the full set.

Table 1. Deterministic-mode agreement between `py-corrected` and `rs-steel` on the published grid, reverse-cumulative along the stellar-mass axis.

Output	median	p90	integral ratio
Satellite SMF	0.93%	23.9%	0.9975
High-z satellite SMF	1.09%	2.4%	0.9959
Satellite SMHM	1.48%	4.4%	0.9959
Merger accretion history	0.70%	27.1%	1.0018
Pair fraction	0.58%	9.9%	1.0001
Accretion redshift	0.94%	11.2%	0.9975

Table 2. Stochastic-mode effect of the defect fixes: `py-as-is` vs `py-corrected`, ensemble means over 5 seeds, published grid, reverse-cumulative.

Output	median	p90	integral ratio
Satellite SMF	5.7%	13.7%	0.9908
High-z satellite SMF	3.0%	72.3%	1.0080
Merger accretion history	**27.1%**	78.1%	0.9942
Pair fraction	11.0%	75.7%	0.9905
Accretion redshift	11.3%	91.8%	0.9908

5.2 Defects found in the original code

Building an independent implementation from the papers' equations, rather than from the code, surfaced defects in the original that the single-implementation development process could not have caught. We group them by consequence; the full list of thirty, including several with no effect on any published number, is maintained alongside the code.

Silently zero pair fractions. The entire pair-fraction accumulation block in `STEEL.py` is nested inside a conditional on the satellite stellar-mass array's dimensionality, with no `else` branch. That array is two-dimensional exactly when satellites are evolved after infall – i.e. exactly when stellar stripping or star formation is switched on. Every configuration Papers 2 and 3 use has one or both switched on, so the pair-fraction output those papers report on was zero for every run built after this code path was (evidently inadvertently) introduced. This is the single most consequential defect in the list.

A one-timestep-short evolution window and a unit mismatch. The satellite post-infall evolution loop runs one timestep short of the window the papers describe, and separately, the star-formation-history accumulator adds a log-ratio quantity into a linear-solar-mass running total. Both bite only when stellar stripping or star formation is on – again, every configuration Papers 2 and 3 use.

A quenching time-scale clamped to a single value. The Fillingham et al. (2016) host-halo-mass dependence of the rapid quenching time-scale is clamped to zero for every host below a galaxy cluster ($M_h < 10^{15}\,M_\odot$), i.e. for most satellites in most runs. This was found while reproducing Paper 1's quenching-time-scale figure during the validation exercise underlying this letter, not by code inspection.

A wrong power spectrum in the accretion history. The halo mass-accretion-history calculation is handed a Harrison–Zel'dovich spectral index ($n_s = 1$) rather than the Planck value used everywhere else in the model, shifting individual halo growth histories by up to 0.069 dex.

A post-processing defect that fed directly into a published pair-fraction figure. Independently of the simulation code, `Return_PF_Plot` in the post-processing script digitizes an upper mass-ratio cut against the wrong array, silently admitting pairs above the intended selection.

Table 2 summarises the effect of these fixes: in stochastic mode, comparing `py-as-is` against `py-corrected`, the integrated satellite stellar mass function moves by under 1% – the model's overall normalisation is robust to every defect found – but the *shape* moves substantially in exactly the outputs Papers 2 and 3 build their headline results on: the merger accretion history's median bin shifts by 27% and the pair fraction by 11%. The same comparison in deterministic mode, py-corrected against rs-steel (Table 1), agrees to under 1% throughout – the disagreement in Table 2 is attributable to the defects, not to any port error, because the port-fidelity comparison that could show a port error does not show one.

5.3 Performance

On the published grid, one process, `rs-steel` completes in 65.6 s against `py-corrected`'s 296.4 s in deterministic mode (68.7 s vs 311.3 s with scatter on) – a 4.5× speedup, consistent between modes. The advantage is largest on frozen-model configurations (no star formation, no stripping), where the Python's per-timestep Cython call still dominates the runtime and the Rust's does not; the single largest contributor was tabulating the halo mass function and virial radius once per (i, j) grid cell rather than re-evaluating the mass-variance integral inside the satellite evolution loop, removing of order 10^8 redundant quadratures, verified bit-identical across all 53 output arrays before and after.

5.4 Reproducing the papers' figures

We rebuilt every results figure in all three papers under the framework of §4; the full per-figure account, including which are exact reproductions, which are partial (typically because one line of a multi-line comparison needs a configuration not otherwise used elsewhere), and which omit only the external data overlay, is maintained online. Figure 1 reproduces Paper 1's headline local satellite stellar mass function, differential and cumulative. Figure 2 reproduces the frozen-model $f_{\rm tdyn}$ and SMHM-evolution sensitivity sweep (Paper 1 Fig. 8) and its associated satellite-distribution grid (Fig. 9), both a four-way, two implementation comparison. Figure 3 reproduces the full three-row central mass-track decomposition (accretion / star formation / total, their fractional and instantaneous-rate contributions) for both the PyMorph and cmodel stellar-mass calibrations (Paper 2 Figs. 6 and 8); the cmodel panel additionally reproduces the paper's own qualitative finding that this calibration's flatter high-mass SMHM slope makes the implied growth history internally inconsistent, requiring negative accretion at some redshifts to balance the stellar mass budget – both implementations agree even through this pathological regime. Figure 4 reproduces Paper 3's central result, the sensitivity of the galaxy pair fraction to each Moster-form SMHM coefficient in turn.

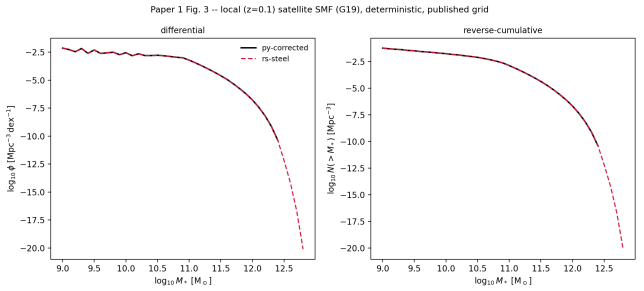

Figure 1. Local ($z \sim 0.1$) satellite stellar mass function, deterministic mode, published grid: differential (left) and reverse-cumulative (right). Solid: `py-corrected`. Dashed: `rs-steel`.

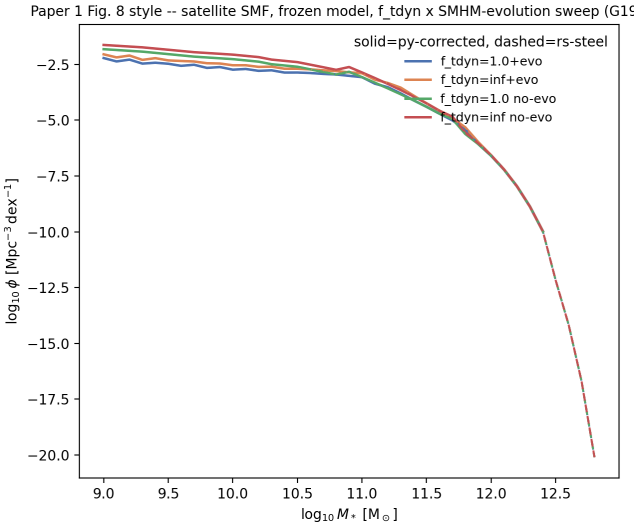

Figure 2. Frozen-model satellite stellar mass function under the $f_{\rm tdyn}\times$SMHM-evolution sweep that Paper 1 §4.1 actually uses (not a stellar-population comparison, which is a different figure in the same paper). Four lines, two implementations.

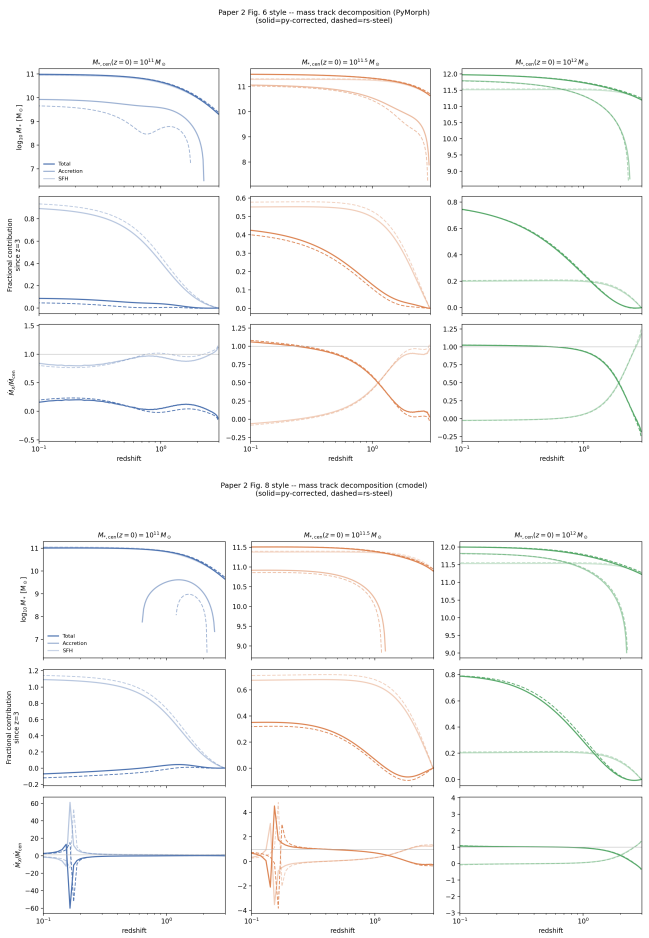

Figure 3. Central mass-track decomposition for the PyMorph (top) and cmodel (bottom) stellar-mass calibrations, three target $z = 0$ masses. Top row of each panel: total, accretion, and in-situ star-formation mass vs. redshift. Middle: fractional contribution to growth since $z = 3$. Bottom: instantaneous accretion-to-total growth-rate ratio. The cmodel panel's $M_* = 10^{11}\ M_\odot$ column shows the growth history becoming internally non-physical, as the published paper itself reports for this calibration.

6 DISCUSSION

The reproducibility failure in §2 is not particular to this code; it is what happens to any unmaintained scientific codebase after enough major-version churn in its dependencies, and semi-empirical models with a handful of users and no packaging discipline are particularly exposed. What is more specific to this exercise is §5.2: every defect we found with a measurable effect on a published figure was found by trying to reproduce that figure from an independent implementation, not by reading the original code, and the most consequential of them (the zeroed pair fraction) is exactly the kind of silent, no-exception, no-warning failure that a single-language, single-implementation test suite built from inside the same codebase is structurally poorly positioned to catch, because the tests and the code share the same blind spot. A from-scratch reimplementation in a different language, with a different numerical stack and a stricter type system, forces every implicit assumption in the original to be stated explicitly somewhere, and the three-way deterministic/stochastic split

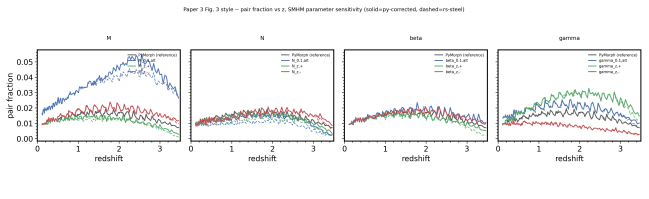

Figure 4. Galaxy pair fraction vs. redshift under thirteen single-coefficient perturbations of the Moster-form SMHM relation (four panels: normalisation, knee mass, low- and high-mass slope), each against a PyMorph reference. Solid: `py-corrected`. Dashed: `rs-steel`. The high-mass slope (γ) and knee-mass (M) panels show the strongest sensitivity, consistent with the published paper's central claim that the high-mass end of the SMHM relation is the dominant systematic in pair-fraction inference.

(§4) lets the resulting disagreements be attributed cleanly to either the port or the original, rather than pooled into one number that means neither.

We do not claim this as a substitute for conventional unit testing – several of the defects in the full list were unreachable code paths with no effect on any output, which a targeted test would have caught more cheaply – but for the specific failure mode of a code path that runs, returns a plausible-looking array, and is wrong, an independent reimplementation checked figure-by-figure against the published results is, in our experience building this one, a more effective net than either code review or unit testing applied after the fact.

7 CONCLUSIONS

We have described a dependency-injected Rust reimplementation of the STEEL semi-empirical galaxy model and validated it against both the originally published code and a defect-corrected version of it, on every headline results figure across the three papers STEEL underlies. The port agrees with a corrected Python implementation to better than 0.5% on every integrated quantity compared in deterministic mode, runs 4.5× faster on the published parameter grid, and its construction surfaced a defect that silently zeroed the pair-fraction output – Paper 3's central observable – in every configuration that paper actually uses. We make the port, the corrected Python branch, and the full defect catalogue available at `https://github.com/PipGrylls/STEEL`.

DATA AVAILABILITY

The Rust port, the corrected Python branch, the validation harness, and every figure and number in this letter are available at `https://github.com/PipGrylls/STEEL`. No new observational data were used.

REFERENCES

Despali G., Giocoli C., Angulo R. E., Tormen G., Sheth R. K., Moscardini L., Baso G., 2016, MNRAS, 456, 2486
Grylls P. J., Shankar F., Zanisi L., Bernardi M., 2019, MNRAS, 483, 2506
Grylls P. J., Shankar F., Leja J., Menci N., Moster B., Behroozi P., Zanisi L., 2020, MNRAS, 491, 634
Grylls P. J., Shankar F., Conselice C., 2020, arXiv:2001.06017
Mundy C. J., Conselice C. J., Duncan K. J., Almaini O., Häußler B., Hartley W. G., 2017, MNRAS, 470, 3507

This paper has been typeset from a TEX/LATEX file prepared by the author.